Second Assignment: Dataflow analysis

Carmine De Rosa
Manuel Gherardi
Arild Kuti

Contents

1 Very Busy Expressions **2**

2 Dominator Analysis **5**

3 Constant Propagation **8**

1 Very Busy Expressions

An expression is defined as a *very busy expression* at a point p if any path starting from p uses the expression without redefining any of its operands.
To identify them, we proceed backward, starting from the *Exit* basic block of the Control Flow Graph (CFG), applying the transfer function and using the intersection ($\cap$) as the meet operator.

Parameter	**Definition**
Domain	Sets of expressions
Direction	Backward: $in[b] = f_b(out[b])$, $out[b] = \wedge in[succ(b)]$
Transfer Function	$in[b] = Gen_b \cup (out[b] \setminus Kill_b)$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$in[\text{Exit}] = \emptyset$
Initial Interior Points	$in[B] = U$ (Universal set)

Table 1: Very Busy Expressions parameters

Algorithm 1 Very Busy Expressions Algorithm

```
in[Exit] ← ∅
for each basic block B ≠ Exit do
    in[B] ← U
end for

changes ← true
while changes do
    changes ← false
    for each basic block B ≠ Exit do
        out[B] ← ⋂_{a∈succ(B)} in[a]
        in_new ← Gen_B ∪ (out[B] \ Kill_B)
        if in[B] ≠ in_new then
            in[B] ← in_new
            changes ← true
        end if
    end for
end while
```

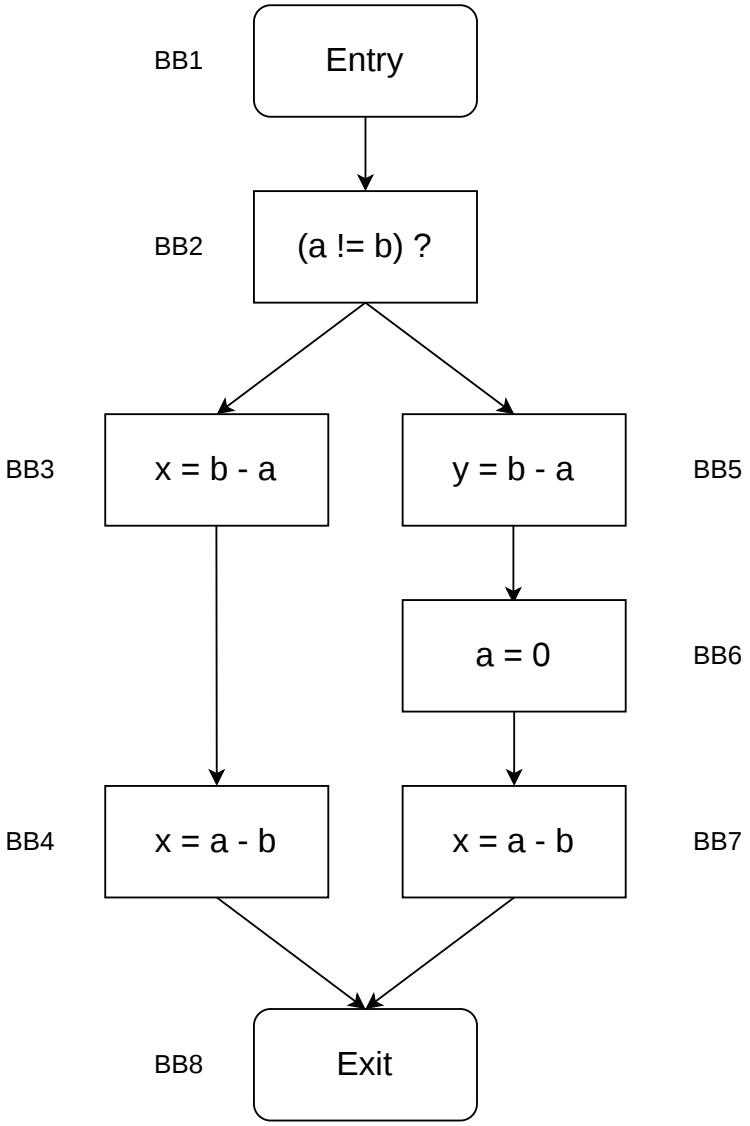

Figure 1: Very busy expression example CFG

	Iteration 1		Iteration 2	
Basic Block	**IN[B]**	**OUT[B]**	**IN[B]**	**OUT[B]**
BB1 (Entry)	$\{b-a\}$	$\{b-a\}$	$\{b-a\}$	$\{b-a\}$
BB2	$\{b-a\}$	$\{b-a\}$	$\{b-a\}$	$\{b-a\}$
BB3	$\{b-a, a-b\}$	$\{a-b\}$	$\{b-a, a-b\}$	$\{a-b\}$
BB4	$\{a-b\}$	$\emptyset$	$\{a-b\}$	$\emptyset$
BB5	$\{b-a\}$	$\emptyset$	$\{b-a\}$	$\emptyset$
BB6	$\emptyset$	$\{a-b\}$	$\emptyset$	$\{a-b\}$
BB7	$\{a-b\}$	$\emptyset$	$\{a-b\}$	$\emptyset$
BB8 (Exit)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Table 2: Iteration table for example CFG (we iterate on BBs in decreasing order)

2 Dominator Analysis

Dominator analysis if essential for building the Static Single Assignment (SSA) form and for finding natural loops in the Control Flow Graph (CFG). A node d dominates a node n if each path from the entry node to n passes through d.

Parameter	**Dominator Analysis**
Domain	Sets of Basic Blocks
Direction	Forward: $out[b] = f_b(in[b]), \quad in[b] = \wedge out[pred[b]]$
Transfer Function	$out[b] = \{b\} \cup in[b]$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$out[\text{Entry}] = \{\text{Entry}\}$
Initial Interior Points	$out[B] = \text{U}$ (Universal set)

Table 3: Dominator Analysis as a Dataflow problem

Algorithm 2 Dominator Analysis Algorithm

```
OUT[Entry] ← {Entry}
for each basic block B ≠ Entry do
    OUT[B] ← U
end for

changes ← true
while changes do
    changes ← false
    for each basic block B ≠ Entry do
        IN[B] ← ⋂_{p∈pred(B)} OUT[p]
        OUT_new ← {B} ∪ IN[B]
        if OUT[B] ≠ OUT_new then
            OUT[B] ← OUT_new
            changes ← true
        end if
    end for
end while
```

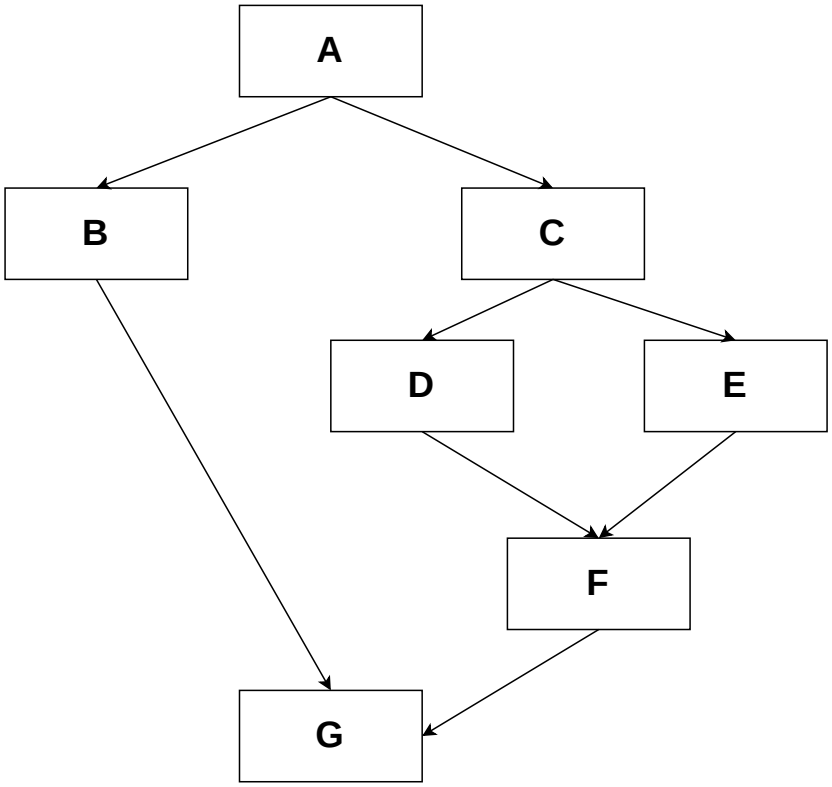

Figure 2: Dominator analysis example CFG

	Iteration 1		**Iteration 2**	
Block	**IN[B]**	**OUT[B]**	**IN[B]**	**OUT[B]**
A (Entry)	$\emptyset$	$\{A\}$	$\emptyset$	$\{A\}$
B	$\{A\}$	$\{A, B\}$	$\{A\}$	$\{A, B\}$
C	$\{A\}$	$\{A, C\}$	$\{A\}$	$\{A, C\}$
D	$\{A, C\}$	$\{A, C, D\}$	$\{A, C\}$	$\{A, C, D\}$
E	$\{A, C\}$	$\{A, C, E\}$	$\{A, C\}$	$\{A, C, E\}$
F	$\{A, C\}$	$\{A, C, F\}$	$\{A, C\}$	$\{A, C, F\}$
G	$\{A\}$	$\{A, G\}$	$\{A\}$	$\{A, G\}$

Table 4: Iteration table for Dominator Analysis on the example CFG.

3 Constant Propagation

Constant Propagation is a global dataflow analysis designed to discover variables that are guaranteed to hold a single constant value at specific program points across all possible execution paths. By tracking pairs of <variable, constant>, this analysis enables the compiler to substitute variable uses with their actual values and statically fold binary expressions whose operands are known constants, thereby optimizing execution efficiency.

Parameter	**Definition**
Domain	Sets of pairs $< variable, value >$
Direction	Forward
	$in[B] = \wedge(out[pred(B)])$
	$out[B] = f_B(in(B)]$
Transfer Function	$f_B(x) = Gen_B \cup (x \setminus Kill_B)$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$out[entry] = \emptyset$
Initial Interior Points	$out[B] = U$ (Universal set)

Table 5: Constant Propagation parameters

Algorithm 3 Constant Propagation

```
out[Entry] ← ∅

for each basic block B ≠ Entry do
    out[B] ← U
end for

changes ← true
while changes do
    changes ← false
    for each basic block B ≠ Entry do
        // Meet operation: combine constants from all predecessors
        in[B] ← ⋂_{p∈pred(B)} out[p]

        // Transfer function: apply constant folding
        out_new ← f_B(in[B])

        if out[B] ≠ out_new then
            out[B] ← out_new
            changes ← true
        end if
    end for
end while
```

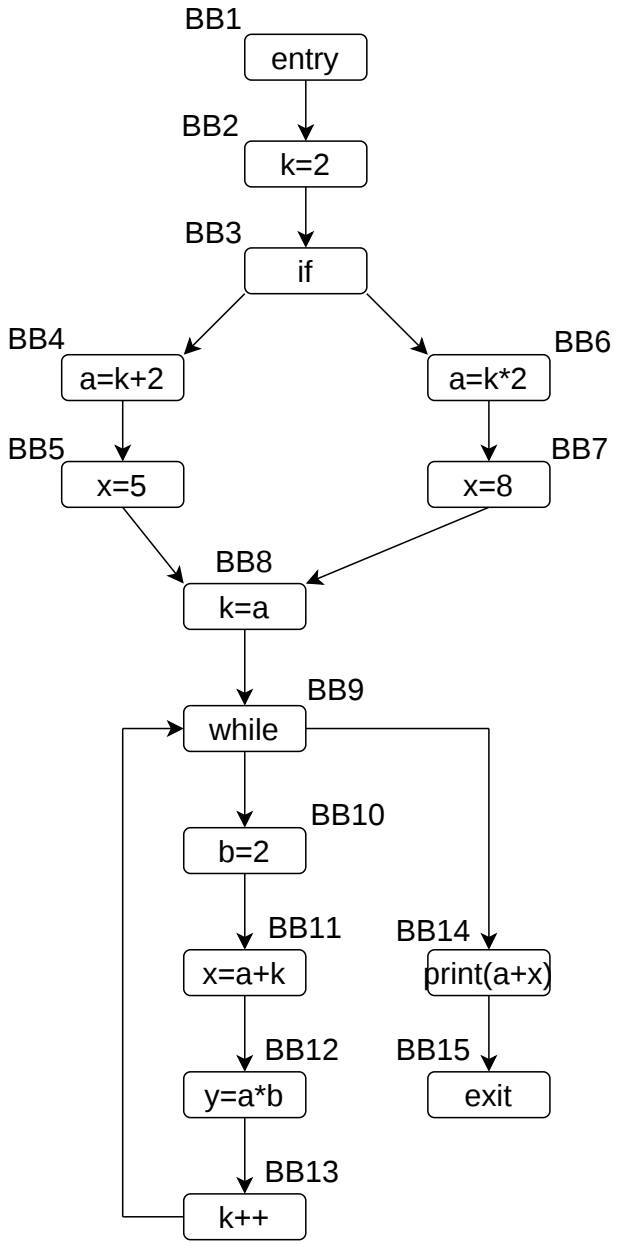

Figure 3: Constant Propagation example CFG

	Iteration 1		Iteration 2	
Basic Block	**IN[B]**	**OUT[B]**	**IN[B]**	**OUT[B]**
BB1 (Entry)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
BB2	$\emptyset$	$< k, 2 >$	$\emptyset$	$< k, 2 >$
BB3	$< k, 2 >$	$< k, 2 >$	$< k, 2 >$	$< k, 2 >$
BB4	$< k, 2 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >$	$< k, 2 >, < a, 4 >$
BB5	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >, < x, 5 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >, < x, 5 >$
BB6	$< k, 2 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >$	$< k, 2 >, < a, 4 >$
BB7	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >, < x, 5 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >, < x, 8 >$
BB8	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$
BB9	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$	$< a, 4 >$	$< a, 4 >$
BB10	$< k, 4 >, < a, 4 >$	$< k, 4 >, < a, 4 >, < b, 2 >$	$< a, 4 >$	$< a, 4 >, < b, 2 >$
BB11	$< k, 4 >, < a, 4 >, < b, 2 >$	$< k, 4 >, < a, 4 >,$ $< b, 2 >, < x, 8 >$	$< a, 4 >, < b, 2 >$	$< a, 4 >, < b, 2 >$
BB12	$< k, 4 >, < a, 4 >,$ $< b, 2 >, < x, 8 >$	$< k, 4 >, < a, 4 >, < b, 2 >,$ $< x, 8 >, < y, 8 >$	$< a, 4 >, < b, 2 >$	$< a, 4 >, < b, 2 >, < y, 8 >$
BB13	$< k, 4 >, < a, 4 >, < b, 2 >,$ $< x, 8 >, < y, 8 >$	$< k, 5 >, < a, 4 >, < b, 2 >,$ $< x, 8 >, < y, 8 >$	$< a, 4 >, < b, 2 >, < y, 8 >$	$< a, 4 >, < b, 2 >, < y, 8 >$
BB14	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$	$< a, 4 >$	$< a, 4 >$
BB15 (Exit)	$< k, 2 >, < a, 4 >$	$< k, 2 >, < a, 4 >$	$< a, 4 >$	$< a, 4 >$

Table 6: Constant Propagation Algorithm Execution Table

The third iteration is equal to the second one so we omitted it.